

Практическое занятие №9: Машинное обучение с scikit-learn III

Ансамбли (Случайный лес), подбор гиперпараметров (GridSearchCV), анализ важности признаков

Цели занятия

1. **Обучающая:** сформировать у студентов практические навыки применения ансамблевых методов классификации, в частности — случайного леса (Random Forest), а также освоить методы автоматического подбора гиперпараметров (GridSearchCV) и интерпретации моделей через анализ важности признаков.
2. **Развивающая:** научить студентов оценивать качество ансамблевых моделей, сравнивать их с одиночными деревьями решений, понимать преимущества ансамблирования в снижении дисперсии и борьбе с переобучением.
3. **Воспитательная:** привить культуру осознанного подбора гиперпараметров, понимание trade-off между вычислительными затратами и качеством модели, а также умение извлекать из модели содержательные выводы о структуре данных.

Образовательные результаты: По завершении занятия студент будет уметь:

- Строить и обучать случайный лес (RandomForestClassifier) с помощью scikit-learn.
- Выполнять подбор гиперпараметров методом GridSearchCV с кросс-валидацией.
- Визуализировать и интерпретировать важность признаков, полученную из случайного леса.
- Сравнить качество случайного леса с одиночным деревом решений и делать выводы о целесообразности применения ансамбля.

Задачи

1. Загрузить датасет и выполнить первичный анализ.
2. Построить и обучить одиночное дерево решений в качестве базового решения.
3. Построить и обучить случайный лес с параметрами по умолчанию.
4. Выполнить подбор гиперпараметров случайного леса с помощью GridSearchCV.
5. Сравнить качество моделей до и после подбора гиперпараметров.
6. Проанализировать важность признаков с помощью встроенного механизма `feature_importances_`.
7. Визуализировать результаты (графики важности признаков, сравнение метрик).
8. Оформить решение в виде Jupyter Notebook с комментариями и выводами.

Теоретические основы

1. Ансамблевые методы машинного обучения

Ансамблевые методы комбинируют несколько моделей (называемых «базовыми» или «слабыми» классификаторами) для получения итогового предсказания, которое, как правило, превосходит предсказание любой отдельной модели. Основная идея ансамблирования описана в работе Бреймана (2001), которая показывает, что объединение многих «нестабильных» моделей позволяет значительно улучшить обобщающую способность алгоритма.

Основные типы ансамблей:

- **Бэггинг (Bagging)** — Bootstrap Aggregating. Обучает несколько моделей на разных подвыборках исходных данных (с возвращением), а затем усредняет их предсказания (или использует голосование). Этот метод особенно эффективен для «нестабильных» алгоритмов, чувствительных к малым изменениям в данных. Классический бэггинг для деревьев решений реализован в `BaggingClassifier`.
- **Бустинг (Boosting)** — последовательное обучение моделей, где каждая следующая исправляет ошибки предыдущей. Примеры: AdaBoost, Gradient Boosting, XGBoost.
- **Стекинг (Stacking)** — комбинирование предсказаний разных моделей с помощью мета-классификатора.

Случайный лес (Random Forest), предложенный Лео Брейманом в 2001 году, является улучшенной версией бэггинга для деревьев решений. В отличие от простого бэггинга, случайный лес вносит дополнительную случайность при построении каждого дерева: на каждом разбиении рассматривается не все множество признаков, а только случайное их подмножество. Это снижает корреляцию между деревьями в ансамбле и улучшает обобщающую способность.

Связь с проблемой bias-variance trade-off

Одиночное дерево решений обладает **низким смещением (bias)**, но **высокой дисперсией (variance)** — небольшие изменения в обучающих данных могут привести к сильно отличающемуся дереву, что проявляется в переобучении. Ансамблевые методы решают проблему высокой дисперсии: бэггинг и случайный лес уменьшают дисперсию, практически не увеличивая смещение, что приводит к более стабильным и точным предсказаниям на новых данных.

Другими словами:

- **Смещение (bias)** — систематическая ошибка модели, её склонность к упрощению.
- **Дисперсия (variance)** — чувствительность модели к малым изменениям в данных.

Случайный лес «усредняет» ошибки множества деревьев, в результате чего случайные отклонения взаимно компенсируются, а систематическая ошибка остаётся на прежнем уровне.

2. Алгоритм Random Forest

2.1. Как строится случайный лес?

Случайный лес для классификации строится следующим образом:

Для $t = 1$ до $n_estimators$:

1. Создать bootstrap-выборку из обучающих данных — случайную выборку объектов с возвращением того же размера, что и исходная выборка.
2. Построить дерево решений на этой bootstrap-выборке, но на каждом узле:
 - Выбрать случайное подмножество признаков размером `max_features`.
 - Найти лучший признак и порог разбиения только среди этого подмножества.

- Дерево растёт до тех пор, пока не будут выполнены условия остановки (например, `min_samples_leaf`).

Итоговое предсказание для нового объекта — голосование (большинство) среди всех деревьев.

2.2. Ключевые параметры RandomForestClassifier

Согласно официальной документации scikit-learn, основные параметры классификатора случайного леса включают:

Параметр	Назначение	Рекомендации по настройке
n_estimators	Количество деревьев в лесу (по умолчанию 100)	Чем больше, тем стабильнее модель, но дольше обучение; хороший баланс — 100-300
max_depth	Максимальная глубина дерева (по умолчанию None — не ограничена)	Ограничение глубины уменьшает переобучение; значения 10-30 для умеренных данных
min_samples_split	Минимальное количество образцов для разбиения узла (по умолчанию 2)	Увеличение уменьшает переобучение; типичные значения: 2, 5, 10
min_samples_leaf	Минимальное количество образцов в листе (по умолчанию 1)	Чем больше, тем более «гладкая» модель; значения 1, 2, 4
max_features	Количество признаков, рассматриваемых для лучшего разбиения (по умолчанию $\sqrt{n \text{ features}}$)	Уменьшение увеличивает разнообразие деревьев; None (все признаки) — больше похоже на бэггинг
bootstrap	Использовать ли bootstrap-выборки (по умолчанию True)	Обычно True; отключение использовать все данные для каждого дерева
criterion	Функция для оценки качества разбиения (по умолчанию gini)	Альтернатива: entropy, log_loss

3. Подбор гиперпараметров с помощью GridSearchCV

GridSearchCV (Grid Search with Cross-Validation) - метод автоматического подбора гиперпараметров путём полного перебора всех комбинаций значений из заданной сетки. Каждая комбинация оценивается с помощью кросс-валидации (обычно 5-кратной), после чего выбирается комбинация, дающая наилучшее значение заданной метрики.

Основные параметры GridSearchCV:

- `estimator` - модель, параметры которой настраиваются.
- `param_grid` - словарь, где ключи — названия гиперпараметров, значения - списки возможных значений.
- `cv` - количество блоков для кросс-валидации (обычно 5).
- `scoring` - метрика, по которой оценивается качество (например, `accuracy`, `f1`, `roc_auc`).
- `verbose` - уровень детализации вывода (1 или 2 для отслеживания процесса).

После выполнения подбора:

- `grid_search.best_params_` - лучшая комбинация гиперпараметров.
- `grid_search.best_score_` - среднее значение метрики на кросс-валидации.
- `grid_search.best_estimator_` - модель с лучшими гиперпараметрами.

Сетка гиперпараметров для случайного леса (классический подход):

```
param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [None, 10, 20],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}
```

При большой размерности данных рекомендуется сначала выполнить грубый поиск по широкому диапазону, а затем - более точную настройку в суженном диапазоне.

4. Анализ важности признаков

Одно из ключевых преимуществ случайного леса - возможность оценить, какие признаки вносят наибольший вклад в предсказание. После обучения модели можно получить доступ к атрибуту `feature_importances_`, который содержит нормализованные оценки важности для каждого признака. Сумма всех важностей равна 1.

Как вычисляется важность признаков (MDI - Mean Decrease in Impurity):

При построении каждого дерева на каждом разбиении фиксируется, насколько уменьшилась неопределённость (нечистота) узла после этого разбиения. Важность признака — это сумма таких уменьшений по всем узлам всех деревьев, где этот признак использовался. Чем чаще признак участвует в разбиениях и чем сильнее он уменьшает нечистоту, тем выше его важность.

Следует учитывать, что **MDI** оценивает важность признаков только внутри используемых данных; для получения более надёжных оценок в ряде случаев применяют перестановочную важность - `.inspection.permutation_importance()`,

которая оценивает падение качества модели при случайном перемешивании значений признака.

Для визуализации важности признаков используется горизонтальная столбчатая диаграмма:

```
importances = model.feature_importances_  
indices = np.argsort(importances)[::-1]  
  
plt.figure(figsize=(10, 6))  
plt.title("Важность признаков")  
plt.bar(range(X.shape[1]), importances[indices], align="center")  
plt.xticks(range(X.shape[1]), [feature_names[i] for i in indices],  
rotation=45)  
plt.tight_layout()  
plt.show()
```

Такой подход позволяет не только повысить доверие к модели, но и выявить неинформативные признаки, которые можно исключить для ускорения вычислений.

Подробные примеры

Пример 1: Базовое обучение случайного леса и сравнение с деревом решений

```
# Шаг 1: Импорт библиотек  
import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt  
import seaborn as sns  
from sklearn.datasets import load_wine  
from sklearn.model_selection import train_test_split  
from sklearn.tree import DecisionTreeClassifier, plot_tree  
from sklearn.ensemble import RandomForestClassifier  
from sklearn.metrics import accuracy_score, classification_report,  
confusion_matrix  
  
# Шаг 2: Загрузка данных  
wine = load_wine()  
X = wine.data  
y = wine.target  
feature_names = wine.feature_names  
target_names = wine.target_names  
  
print(f"Размер данных: {X.shape}")  
print(f"Классы: {target_names}")  
print(f"Распределение классов:\n{pd.Series(y).value_counts()}")  
  
# Шаг 3: Разделение на выборки  
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=42, stratify=y  
)  
  
# Шаг 4: Одиночное дерево решений (baseline)  
tree = DecisionTreeClassifier(random_state=42)  
tree.fit(X_train, y_train)  
y_pred_tree = tree.predict(X_test)
```

```

tree_accuracy = accuracy_score(y_test, y_pred_tree)

# Шаг 5: Случайный лес (параметры по умолчанию)
rf_default = RandomForestClassifier(random_state=42)
rf_default.fit(X_train, y_train)
y_pred_rf_default = rf_default.predict(X_test)
rf_default_accuracy = accuracy_score(y_test, y_pred_rf_default)

# Шаг 6: Сравнение результатов
print("=== Сравнение моделей ===")
print(f"Одиночное дерево решений: Accuracy = {tree_accuracy:.3f}")
print(f"Случайный лес (по умолчанию): Accuracy = {rf_default_accuracy:.3f}")

# Шаг 7: Визуализация важности признаков для случайного леса
importances = rf_default.feature_importances_
indices = np.argsort(importances)[::-1]

plt.figure(figsize=(10, 6))
plt.title("Важность признаков (Random Forest, параметры по умолчанию)")
plt.bar(range(len(importances)), importances[indices], align="center")
plt.xticks(range(len(importances)), [feature_names[i] for i in indices],
rotation=45)
plt.tight_layout()
plt.show()

# Шаг 8: Матрица ошибок
cm = confusion_matrix(y_test, y_pred_rf_default)
plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=target_names, yticklabels=target_names)
plt.xlabel('Предсказано')
plt.ylabel('Истинное значение')
plt.title('Матрица ошибок — Random Forest (по умолчанию)')
plt.show()

```

Ожидаемый результат: Случайный лес обычно показывает точность выше, чем одиночное дерево (на датасете Wine — около 0.97–1.00 против 0.92–0.97 у дерева). Это подтверждает теорию об уменьшении дисперсии при ансамблировании.

Пример 2: Подбор гиперпараметров с GridSearchCV

```

from sklearn.model_selection import GridSearchCV

# Шаг 1: Определение сетки гиперпараметров
param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [None, 10, 20],
    'min_samples_split': [2, 5],
    'min_samples_leaf': [1, 2]
}

# Шаг 2: Создание и настройка GridSearchCV
rf = RandomForestClassifier(random_state=42)
grid_search = GridSearchCV(
    estimator=rf,
    param_grid=param_grid,
    cv=5,
    scoring='accuracy',
    verbose=1,
    n_jobs=-1
    # 5-кратная кросс-валидация
    # метрика оценки
    # вывод процесса
    # использовать все ядра процессора
)

```

```

# Шаг 3: Обучение с подбором параметров
grid_search.fit(X_train, y_train)

# Шаг 4: Анализ результатов
print("=== Результаты GridSearchCV ===")
print(f"Лучшие гиперпараметры: {grid_search.best_params}")
print(f"Лучшая точность на кросс-валидации: {grid_search.best_score:.3f}")

# Шаг 5: Оценка лучшей модели на тестовой выборке
best_rf = grid_search.best_estimator_
y_pred_best = best_rf.predict(X_test)
best_accuracy = accuracy_score(y_test, y_pred_best)
print(f"Точность лучшей модели на тестовой выборке: {best_accuracy:.3f}")

# Шаг 6: Сравнение с моделью по умолчанию
print(f"\n=== Итоговое сравнение ===")
print(f"Параметры по умолчанию: {rf_default_accuracy:.3f}")
print(f"После GridSearchCV: {best_accuracy:.3f}")
print(f"Улучшение: {best_accuracy - rf_default_accuracy:.3f}")

# Шаг 7: Детальный отчёт по лучшей модели
print("\n=== Classification Report (лучшая модель) ===")
print(classification_report(y_test, y_pred_best, target_names=target_names))

```

Ожидаемый результат: Подбор гиперпараметров может дать небольшое, но значимое улучшение точности (обычно +0.01–0.03).

Пример 3: Глубокий анализ важности признаков

```

# Шаг 1: Получение важности признаков из лучшей модели
importances = best_rf.feature_importances_
feature_importance_df = pd.DataFrame({
    'Признак': feature_names,
    'Важность': importances
}).sort_values('Важность', ascending=False)

print("=== Важность признаков (топ-5) ===")
print(feature_importance_df.head())

# Шаг 2: Визуализация важности признаков
plt.figure(figsize=(12, 6))
sns.barplot(data=feature_importance_df, x='Важность', y='Признак')
plt.title('Важность признаков в случайном лесе')
plt.xlabel('Важность')
plt.tight_layout()
plt.show()

# Шаг 3: Создание упрощённой модели только с топ-3 признаками
top_features = feature_importance_df.head(3)['Признак'].values
indices_top = [list(feature_names).index(f) for f in top_features]

X_train_top = X_train[:, indices_top]
X_test_top = X_test[:, indices_top]

rf_top = RandomForestClassifier(**grid_search.best_params_, random_state=42)
rf_top.fit(X_train_top, y_train)
y_pred_top = rf_top.predict(X_test_top)
accuracy_top = accuracy_score(y_test, y_pred_top)

print(f"Точность модели с топ-3 признаками: {accuracy_top:.3f}")
print(f"Точность полной модели: {best_accuracy:.3f}")
print(f"Снижение точности: {best_accuracy - accuracy_top:.3f}")

```

Ожидаемый результат: Топ-3 признака могут обеспечить точность, лишь незначительно уступающую полной модели (например, 0.94 против 0.97), что указывает на возможность сокращения размерности.

Варианты для самостоятельного выполнения (10 вариантов)

Для каждого варианта студент должен:

1. Загрузить указанный датасет (все варианты используют датасеты, входящие в библиотеку `scikit-learn`).
2. Выполнить разбиение на обучающую (80%) и тестовую (20%) выборки с `random_state=42` и `stratify=y`.
3. Построить одиночное дерево решений (`DecisionTreeClassifier`) в качестве baseline.
4. Построить случайный лес (`RandomForestClassifier`) с параметрами по умолчанию.
5. Выполнить подбор гиперпараметров случайного леса с помощью `GridSearchCV` (минимальная сетка: `n_estimators` [50, 100, 200], `max_depth` [None, 10, 20], `min_samples_split` [2, 5]).
6. Сравнить точность (ассигасу) трёх моделей: дерево решений, случайный лес по умолчанию, случайный лес после подбора.
7. Построить график важности признаков (горизонтальная столбчатая диаграмма).
8. Сделать вывод: какие признаки оказались наиболее важными для предсказания в данной задаче?
9. Сформулировать итоговый вывод: стоит ли применять случайный лес вместо дерева в данной задаче и почему?

Вариант 1: Вина (wine)

- **Датасет:** `load_wine()` из `sklearn.datasets`
- **Тип задачи:** Многоклассовая классификация (3 класса)
- **Количество признаков:** 13 (химические характеристики вина)
- **Цель:** Определение сорта вина
- **Особенность:** визуализировать важность признаков и определить, какой химический компонент вносит наибольший вклад в классификацию сортов вина.

Вариант 2: Ирисы Фишера (iris)

- **Датасет:** `load_iris()` из `sklearn.datasets`
- **Тип задачи:** Многоклассовая классификация (3 класса)
- **Количество признаков:** 4 (длина и ширина чашелистика и лепестка)
- **Цель:** Классификация сорта ириса
- **Особенность:** сравнить важность признаков — какой из четырёх параметров наиболее информативен для разделения видов?

Вариант 3: Цифры (digits)

- **Датасет:** `load_digits()` из `sklearn.datasets`
- **Тип задачи:** Многоклассовая классификация рукописных цифр (10 классов)
- **Количество признаков:** 64 (8×8 изображения, значения яркости пикселей от 0 до 16)
- **Цель:** Распознавание рукописной цифры
- **Особенность:** визуализировать важность признаков в виде тепловой карты 8×8 — какие пиксели изображения наиболее важны для распознавания?

Вариант 4: Игрушечный датасет для классификации (make_classification)

- **Датасет:** `make_classification(n_samples=1000, n_features=20, n_informative=5, n_redundant=3, random_state=42)` — синтетический датасет, где только 5 из 20 признаков действительно информативны
- **Тип задачи:** Бинарная классификация
- **Цель:** Классификация объектов по синтезированным признакам
- **Особенность:** проанализировать важность признаков и определить, сколько действительно информативных признаков было выделено моделью.

Вариант 5: Даты (fetch_california_housing, преобразованный в классификацию)

- **Датасет:** `fetch_california_housing()` — задача регрессии; преобразовать целевую переменную в бинарную: ниже и выше медианы
- **Тип задачи:** Бинарная классификация (дорого/недорого)
- **Количество признаков:** 8 (медианный доход, возраст дома, количество комнат и т.д.)
- **Цель:** Прогнозирование, относится ли дом к категории «дорогих» (выше медианы)
- **Особенность:** Целевая переменная должна быть преобразована перед обучением; сравнить важность признаков с доменами знаний рынка недвижимости.

Вариант 6: Пингвины Палмера (fetch_openml, id=42585)

- **Датасет:** `fetch_openml(data_id=42585)` — датасет Palmer Penguins
- **Тип задачи:** Многоклассовая классификация (3 вида пингвинов)
- **Количество признаков:** Набор признаков включает длину и глубину клюва, длину ласт, массу тела и пол
- **Цель:** Классификация вида пингвина по биометрическим признакам
- **Особенность:** обработать пропущенные значения (если есть) перед обучением; проанализировать, какие биометрические параметры наиболее важны.

Вариант 7: Диабет с преобразованием (load_diabetes, бинарная классификация)

- **Датасет:** `load_diabetes()` — задача регрессии; преобразовать целевую переменную в бинарную (прогрессирование заболевания выше/ниже медианы)
- **Тип задачи:** Бинарная классификация
- **Количество признаков:** 10 (возраст, пол, ИМТ, артериальное давление и т.д.)
- **Цель:** прогнозирование высокого уровня прогрессирования диабета по клиническим показателям

- **Особенность:** выполнить масштабирование признаков (StandardScaler) и сравнить, изменилась ли важность признаков после масштабирования.

Вариант 8: Изображения молекул MNIST (fetch_openml, id=179)

- **Датасет:** `fetch_openml(data_id=179)` — датасет молекулярных изображений MNIST (аналогичен `digits`, но с другими объектами)
- **Тип задачи:** Многоклассовая классификация
- **Количество признаков:** 64 (эмпирические)
- **Цель:** Классификация изображений
- **Особенность:** использовать только первые 5000 образцов для ускорения вычислений; визуализировать важность признаков.

Вариант 9: Геологические породы (fetch_openml, data_id=44)

- **Датасет:** `fetch_openml(data_id=44)` — датасет «Spectf heart» или иной подходящий датасет без ограничений по условию
- **Тип задачи:** Задача, требующая очистки
- **Цель:** Классификация после предобработки
- **Особенность:** выполнить полный цикл предобработки (обработка пропусков, удаление выбросов) перед обучением случайного леса.

Вариант 10: Титаник (titanic, если доступен, или fetch_openml)

- **Датасет:** при наличии доступа — встроенный датасет «Титаник»; иначе — синтетический датасет с категориальными признаками
- **Тип задачи:** Бинарная классификация (выжил/не выжил)
- **Особенность:** выполнить кодирование категориальных признаков (OneHotEncoder) перед обучением; проанализировать, какие признаки оказались наиболее важными для прогнозирования выживания.

Контрольные вопросы для самопроверки

1. **Что такое ансамблевый метод и зачем он нужен?** Объясните на примере случайного леса.
 - *Ответ:* Ансамблевый метод комбинирует несколько моделей для получения лучшего предсказания, чем у каждой модели по отдельности. Случайный лес — это ансамбль деревьев решений, который снижает дисперсию и улучшает обобщающую способность.
2. **Чем бэггинг отличается от бустинга?** Приведите ключевые различия.
 - *Ответ:* Бэггинг обучает модели параллельно на независимых подвыборках (с бутстрепом), затем усредняет их предсказания, что уменьшает дисперсию. Бустинг обучает модели последовательно, каждая следующая исправляет ошибки предыдущей, что уменьшает смещение. Случайный лес относится к бэггингу.
3. **Как случайный лес уменьшает дисперсию по сравнению с одиночным деревом?**
 - *Ответ:* Усреднение предсказаний многих деревьев, каждое из которых может «переобучаться» по-своему, приводит к тому, что случайные ошибки

взаимно компенсируются. При этом систематическая ошибка (смещение) остаётся на уровне отдельного дерева.

4. **Каков смысл параметра `n_estimators`? Что произойдёт, если его увеличить?**
 - *Ответ:* `n_estimators` — количество деревьев в лесу. При увеличении этого параметра модель становится более стабильной (уменьшается дисперсия), но растёт время обучения и предсказания. Начиная с некоторого порога (обычно 100–300), дальнейшее увеличение даёт лишь незначительный прирост качества.
5. **Почему для случайного леса характерна меньшая склонность к переобучению, чем для глубинного дерева решений?**
 - *Ответ:* Каждое дерево в лесу растётся глубоко (часто без ограничений), но их ансамбль усредняет ошибки, что предотвращает общее переобучение. Введение случайности (ограничение признаков на каждом разбиении) также уменьшает корреляцию между деревьями и улучшает обобщение.
6. **Как в `scikit-learn` получить важность признаков после обучения случайного леса? Как интерпретировать эти значения?**
 - *Ответ:* используется атрибут `feature_importances_`. Большее значение означает, что признак чаще использовался в узлах деревьев и сильнее уменьшал нечистоту. Сумма всех важностей равна 1. Высокая важность указывает на то, что признак информативен для разделения классов.
7. **Что такое кросс-валидация и почему она используется в `GridSearchCV`?**
 - *Ответ:* Кросс-валидация разбивает данные на k блоков, обучает модель на $k-1$ блоке и проверяет на оставшемся. Она позволяет более надёжно оценить качество модели и избежать переобучения на конкретном разбиении. `GridSearchCV` использует кросс-валидацию для объективной оценки каждой комбинации гиперпараметров.
8. **В чём различие между `GridSearchCV` и `RandomizedSearchCV`? Когда какой метод предпочтительнее?**
 - *Ответ:* `GridSearchCV` перебирает все комбинации заданной сетки, что гарантирует нахождение лучшего варианта, но может быть очень медленным при большом количестве комбинаций. `RandomizedSearchCV` случайным образом выбирает ограниченное число комбинаций, что быстрее и часто даёт результат, близкий к оптимальному. Предпочтительнее использовать `GridSearchCV` при небольшом пространстве гиперпараметров (до 100–200 комбинаций) и `RandomizedSearchCV` при большом и многомерном пространстве.
9. **Какой параметр случайного леса ограничивает максимальную глубину деревьев? Как он влияет на переобучение?**
 - *Ответ:* Параметр `max_depth`. Ограничение глубины делает деревья менее сложными, что уменьшает переобучение, но может увеличить смещение (недообучение). Значение `None` (по умолчанию) означает отсутствие ограничения — деревья растут до полной чистоты, что может приводить к переобучению.
10. **Случайный лес выдаёт не только класс, но и вероятности. Какую функцию для этого нужно использовать и где это может быть полезно?**
 - *Ответ:* Метод `predict_proba()` возвращает вероятности принадлежности к каждому классу. Это полезно для задач, где важно не только предсказание класса, но и степень уверенности модели (например, в системах поддержки принятия решений, ранжировании по риску и т.п.).

Дополнительные источники информации и литература

1. **Официальная документация scikit-learn: RandomForestClassifier** — все параметры и методы.
<https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html>
2. **Официальная документация GridSearchCV**.
https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GridSearchCV.html
3. **Пример работы со случайным лесом и важностью признаков от scikit-learn**.
https://scikit-learn.org/stable/auto_examples/ensemble/plot_forest_importances.html
4. **Сравнение GridSearchCV и RandomizedSearchCV** (официальный пример).
https://scikit-learn.org/stable/auto_examples/model_selection/plot_randomized_search.html
5. **Статья «Random Forests» (Breiman, 2001)** — оригинальная публикация.
Доступна по DOI: 10.1023/A:1010933404324
6. **Книга «Python и анализ данных» (Уэс Маккинни)** — главы по ансамблевым методам.
7. **Курс по машинному обучению на Stepik** — разделы по случайному лесу и подбору гиперпараметров.
<https://stepik.org/course/4852>
8. **Блог ODS (Открытая наука о данных)** — статьи по интерпретации важности признаков.
<https://habr.com/ru/company/ods/blog/>

Критерии оценивания (100-балльная шкала)

№	Критерий	Баллы	Комментарий
1.	Загрузка и первичный анализ данных	8	Загрузка датасета, вывод <code>.shape</code> , <code>.info()</code> , <code>.head()</code> , распределение классов
2.	Разделение на выборки (<code>train_test_split</code>)	6	Правильное выделение признаков и целевой переменной; использование <code>stratify=y</code> и <code>random state</code>
3.	Одиночное дерево решений (<code>baseline</code>)	10	Корректное обучение и вычисление метрик (<code>accuracy</code> , <code>classification report</code>)
4.	Случайный лес (параметры по умолчанию)	10	Корректное обучение и оценка качества
5.	Подбор гиперпараметров с <code>GridSearchCV</code>	20	Правильное определение сетки параметров, использование кросс-валидации, извлечение лучшей модели
6.	Сравнительный анализ качества моделей	12	Сравнение ассигасу дерева, RF по умолчанию и RF после подбора; содержательный вывод
7.	Анализ важности признаков	15	Построение графика важности, содержательный анализ и интерпретация топ-признаков
8.	Оформление кода и блокнота	9	Комментарии, логическая структура ячеек, наличие Markdown-заголовков, чистый вывод
9.	Использование системы контроля версий Git (<code>push</code> на GitHub)	+5	Бонусные баллы за загрузку работы в удалённый репозиторий
10.	Дополнительные выводы (сравнение с упрощённой моделью из топ-признаков)	5	Бонус за расширенный анализ
	ИТОГО	95+5(+5)	

Шкала перевода баллов в оценку

Баллы	Оценка
90–100	отлично
75–89	хорошо
60–74	удовлетворительно
менее 60	неудовлетворительно